

Herald



Protocol Research: Apache Kafka

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only (no implementation yet)
Status summary	Apache Kafka is the dominant distributed log + pub/sub system (LinkedIn 2011 → Apache TLP). Wire protocol: binary, framed, TCP-based. For Herald, Kafka is a P3 INGRESS protocol: Herald consumes Kafka topics as event streams. The CloudEvents-over-Kafka binding (Herald's existing CloudEvents path lights up cleanly). Two Go libraries: <code>segmentio/kafka-go</code> (pure Go) or <code>confluent-kafka-go</code> (cgo wrapper around <code>librdkafka</code>). Both viable; <code>segmentio</code> for pure-Go-simplicity; <code>confluent</code> for production hardening. P3 opt-in.
Issues	open-questions: <code>segmentio</code> vs <code>confluent</code> ; per-tenant consumer groups; partition-key from <code>tenant_id</code> .
Continuation	Wave 4f+ — open HRD-335..HRD-344 only if operator wants Kafka ingest.

Constitutional anchors

- **§107 anti-bluff** — tests boot real Kafka (single-broker via `containers/`); real `kafka-console-producer` publishes; Herald consumes; assert receive.
- **§11.4.74 catalogue-check** — no `vasic-digital` or `HelixDevelopment` Kafka module. Use `segmentio/kafka-go` (pure Go) — recommended.
- **§11.4.61** — tracked doc.

Table of contents

- [§1. Protocol overview](#)
- [§2. Specification deep-dive](#)
- [§3. Herald-specific applicability analysis](#)
- [§4. Step-by-step implementation guide for Herald](#)
- [§5. §107 anti-bluff testing strategy](#)
- [§6. Open questions for operator](#)
- [§7. References](#)
- [§8. Catalogue-check verdict](#)

§1. Protocol overview

Apache Kafka. Distributed event-streaming platform. Wire protocol: custom binary, TCP-based, frame-oriented. Versioned via "API versions" + "broker version" handshake.

Topology. Producers → topics → partitions → consumers. Consumers organize in consumer GROUPS for load balancing.

Durability. Each partition is an append-only log; replicated across N brokers (default 3); messages persist for `retention.ms` (e.g. 7 days) regardless of consumption.

Delivery semantics.

- **At-most-once** — manual offset commit BEFORE processing.
- **At-least-once** — manual offset commit AFTER processing.
- **Exactly-once** — transactional producer + `read_committed` consumer + idempotent producer.

Authentication. SASL (PLAIN, SCRAM, OAUTHBEARER, GSSAPI/Kerberos), mTLS, or plain.

Authorization. ACLs per topic/group/consumer.

Adoption. Pervasive in big-data ecosystems. LinkedIn (origin), Netflix, Uber, Airbnb, Stripe, etc. Also Confluent Cloud (managed), AWS MSK, Azure Event Hubs (Kafka API), Redpanda (Kafka-compatible).

§2. Specification deep-dive

§2.1 Records

Each record: `{ key: bytes, value: bytes, headers: [{k, v}], timestamp, offset, partition }`. headers arrived in Kafka 0.11; CloudEvents bindings live there.

§2.2 Partitioning

Producer hashes the key (or specifies partition) → routes to a specific partition. Within a partition, order is preserved. Across partitions, no order guarantee.

§2.3 Consumer offsets

Consumer tracks the next offset to read per partition. Offsets are stored in a special Kafka topic `__consumer_offsets`. Modern API: `auto.commit.enable=false`; commit after processing.

§2.4 Consumer groups

Consumers in the same group share partitions; one consumer reads each partition. Adding consumers → rebalance.

§2.5 Idempotent producer

`enable.idempotence=true` ensures retries don't create duplicates (PID + sequence number per partition).

§2.6 Transactional producer

`transactional.id + init_transactions() + begin_transaction() / commit_transaction()` for atomic multi-partition writes.

§2.7 CloudEvents Kafka binding

CloudEvents v1.0.2 → Kafka headers prefixed `ce_`. Value = CloudEvent data. Partition key from `partitionkey` extension attribute.

§3. Herald-specific applicability analysis

§3.1 Use case

Herald INGESTS Kafka topics. Example:

- Microservice X publishes audit logs to `audit.events.v1` topic.
- Herald subscribes; each record becomes an event.

Compared to MQTT: Kafka is higher-throughput, durable, replay-able; MQTT is lighter, IoT-friendly.

§3.2 Herald is a CLIENT

No Kafka broker in Herald. Operator provides existing Kafka.

§3.3 segmentio/kafka-go vs confluent-kafka-go

Aspect	segmentio	confluent
Implementation	pure Go	cgo wrapper around librdkafka (C)
Performance	good	excellent (battle-tested at scale)
context.Context	yes	partial
Builds without C toolchain	yes	NO (cgo)
Maintainer	Twilio Segment (acquired)	Confluent
Production hardening	good	best-in-class

Recommendation: segmentio/kafka-go for Herald — Herald is not a billion-msg/day platform; pure-Go simplicity beats marginal performance.

§3.4 Multi-tenant

Two approaches:

1. **Per-tenant consumer group** — each tenant has its own consumer group. Clean isolation.

2. **Shared consumer + tenant in header** — single Herald consumer ingests all tenants' events; routes via `ce_tenant_id` header.

Recommend approach 1.

§3.5 Partition key from `tenant_id`

Producer side: when Herald PUBLISHES to Kafka, set `key = tenant_id`. This ensures all events from one tenant land in the same partition (order preserved per-tenant).

§3.6 Idempotency

`message.id` (CloudEvents `ce_id` header) is the idempotency key. Same Redis SETNX path.

§3.7 OTel propagation

CloudEvents `ce_traceparent` header → Herald extracts.

§3.8 Failure modes

1. **Rebalance pause.** During rebalance, no consumption; events delayed. Monitor; tune `session.timeout.ms`.
2. **Offset commit lag.** If Herald crashes before committing, replays from last commit → duplicates; idempotency handles.
3. **Lag.** Monitor `consumer-lag` per partition.
4. **Big-msg.** Kafka default 1 MiB per record; Herald must handle this limit gracefully.

§4. Step-by-step implementation guide for Herald

§4.1 Add dep

Submodule: `github.com/segmentio/kafka-go` under `submodules/go-kafka/`.

§4.2 New channel adapter

`commons_messaging/channels/kafka/` — Channel interface; ingest + egress.

§4.3 Subscriber config

- `kafka_brokers` — comma-separated.
- `kafka_topic`, `kafka_consumer_group`.
- `kafka_sasl_mechanism` (PLAIN/SCRAM/OAUTHBEARER).
- `kafka_username` / `kafka_password` (encrypted).
- `kafka_tls_enabled`.

§4.4 Egress

Herald publishes notifications to Kafka topics (for downstream consumers — log aggregators, SIEMs).

§4.5 New e2e_bluff_hunt invariants

- **E103** — boot Kafka via `containers/; kafka-console-producer.sh --topic test --message '{...}'`; Herald consumes; `events_processed` row.
- **E104** — CloudEvents binding: produce with `ce_*` headers; assert `ce-id` propagates.
- **E105** — offset commit after processing: kill Herald mid-process; restart; assert no message loss (resume from committed offset).
- **E106** — exactly-once-effective: produce duplicate message; idempotency dedupes via `ce_id`.
- **E107** — consumer-group rebalance: scale Herald horizontally; partitions redistributed.

§4.6 HRD scaffolding

- **HRD-335** — vendor `segmentio/kafka-go`.
- **HRD-336** — channel adapter `commons_messaging/channels/kafka/`.
- **HRD-337** — consumer group manager.
- **HRD-338** — CloudEvents Kafka binding.
- **HRD-339** — egress producer.
- **HRD-340** — SASL auth options.
- **HRD-341** — offset commit policy.
- **HRD-342** — `e2e_bluff_hunt` E103–E107 + mutation gates.
- **HRD-343** — spec V3 §4t amendments.
- **HRD-344** — close Wave 4f (Kafka slice).

§5. §107 anti-bluff testing strategy

§5.1 Happy-path

- **Test 1: consume.** Single-broker Kafka up via `containers/; kafka-console-producer.sh` publishes; Herald consumes; `events_processed` row.
- **Test 2: produce.** Herald notify `kafka` → `kafka-console-consumer.sh` receives.
- **Test 3: CloudEvents binding.** `ce_id`, `ce_source` headers preserved.

§5.2 Reliability

- **Test 4: at-least-once.** Crash Herald after processing but before commit → on restart, message redelivered; Redis SETNX dedupes.
- **Test 5: idempotent producer.** Set `enable_idempotence=true`; force retry; assert single record at broker.

§5.3 Failure modes

- **Test 6: broker disconnect.** Stop+start Kafka; consumer reconnects.
- **Test 7: bad credentials.** SASL fails; backoff.
- **Test 8: consumer rebalance.** Start 2nd Herald instance; partitions split.

§5.4 Performance

- **Test 9: throughput.** 50K records/sec sustained; lag < 5s.

§5.5 Mutation gates

- Mutation: skip offset commit → Test 4 FAILS (loses progress).
- Mutation: skip ce_id propagation → Test 3 FAILS.

§5.6 Wire-level

- tcpdump :9092 → Kafka protocol frames; assert FETCH + OFFSET_COMMIT sequence.

§6. Open questions for operator

1. **In scope for MVP?** Recommend NO.
2. **segmentio or confluent?** Recommend segmentio.
3. **Per-tenant consumer group?** Yes.
4. **Egress also?** Yes (cheap once consumer lib is in).
5. **Exactly-once transactions worth it?** Recommend NO; at-least-once + idempotency is simpler.
6. **Reuse containers/ for Kafka tests?** Yes (single-broker Kafka image).

§7. References

(All fetched 2026-05-22.)

- **Kafka protocol.** <https://kafka.apache.org/protocol>
- **Kafka docs.** <https://kafka.apache.org/documentation/>
- **segmentio/kafka-go.** <https://github.com/segmentio/kafka-go>
- **confluent-kafka-go.** <https://github.com/confluentinc/confluent-kafka-go>
- **librdkafka.** <https://github.com/confluentinc/librdkafka>
- **CloudEvents Kafka binding.** <https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/bindings/kafka-protocol-binding.md>
- **cloudevents/sdk-go Kafka protocol.** https://github.com/cloudevents/sdk-go/tree/main/protocol/kafka_sarama
- **Confluent Go client docs.** <https://docs.confluent.io/kafka-clients/go/current/overview.html>
- **2026 production guide.** <https://medium.com/@harishsingh8529/using-kafka-with-go-read-this-before-you-push-to-production-d6d64ce60e17>

§8. Catalogue-check verdict

- **vasic-digital:** no module.
- **HelixDevelopment:** no module.

Verdict: no-match → **vendor** `github.com/segmentio/kafka-go` **as submodule.**

Alternative `cloudevents/sdk-go/protocol/kafka_sarama` for CloudEvents-native pattern; choose at implementation time.